

Document Number: P0290R3
Date: 2023-01-06
Author: [Anthony Williams](#)
Audience: SG1

P0290R3: `apply()` for `synchronized_value<T>`

Introduction

This paper is a followup to [P0290R2](#), based on feedback from the Kona 2022 meeting.

The basic idea is that `synchronized_value<T>` stores a value of type `T` and a mutex. The `apply()` function then provides a means of accessing the stored value with the mutex locked, automatically unlocking the mutex afterwards.

The `apply()` function is variadic, so you can operate on a set of `synchronized_value<T>` objects. All the mutexes are locked prior to invoking the supplied callable object, and then they are all released afterwards.

The name is chosen to fit in with `std::apply` for tuples, since the operation is conceptually similar. Rather than expanding a `std::tuple` to supply the arguments to the function, the values wrapped by the `synchronized_values` are extracted to supply the arguments to the function.

This provides an easy way for developers to ensure that all accesses to a given object are done with the relevant mutex locked, whilst also allowing for operations that require locks on multiple objects.

In order to avoid simple mistakes when using the `synchronized_value<T>` objects, there are no public member functions or operations other than construction.

The actual implementation may use an alternative synchronization mechanism instead of a mutex, provided that the synchronization requirements are met.

Examples

1: Simple accesses

Simple accesses can be done with simple lambdas:

```
synchronized_value<std::string> s;  
  
std::string read_value(){  
    return apply([](auto& x){return x;},s);  
}  
  
void set_value(std::string const& new_val){  
    apply([&](auto& x){x=new_val;},s);  
}
```

2: More complex processing

More complex processing can be done with a more complex lambda, or a separate function or callable object:

```
synchronized_value<std::queue<message_type>> queue;  
  
void process_message(){  
    std::optional<message_type> local_message;  
    apply([&](std::queue<message_type>& q){  
        if(!q.empty()){  
            local_message.emplace(std::move(q.front()));  
            q.pop_front();  
        }  
    },queue);  
    if(local_message)  
        do_processing(local_message.value());  
}
```

3: Multi-value processing

The variadic nature of `apply()` means that writing code that accesses multiple `synchronized_value<T>` objects is straightforward. It uses the same mechanism as `std::lock()` to ensure that the requisite mutexes are locked without deadlock.

The ubiquitous example of transferring money between accounts can then be simply written as follows:

```
void transfer_money(
    synchronized_value<account>& from_,
    synchronized_value<account>& to_,
    money_value amount){
    apply([=](auto& from,auto& to){
        from.withdraw(amount);
        to.deposit(amount);
    },from_,to_);
}
```

Proposed wording

Add a new section to chapter 30 as follows.

30.x Synchronized Values

This section describes a class template to provide locked access to a value in order to facilitate the construction of race-free programs.

Header `<synchronized_value>` synopsis

```
namespace std {
    template<class T>
    class synchronized_value;

    template<class F,class FirstValue,class ... OtherValues>
    decltype(auto) apply(
        F&& f,synchronized_value<FirstValue>& first_value,synchronized_value<ValueTypes>&... other_values);
}
```

30.x.1 Class template `synchronized_value`

```
namespace std
{
    template<class T>
    class synchronized_value
    {
    public:
        synchronized_value(synchronized_value const&) = delete;
        synchronized_value& operator=(synchronized_value const&) = delete;

        template<class ... Args>
        synchronized_value(Args&& ... args);
        ~synchronized_value();

    private:
        T value; // exposition only
        std::mutex _mut; // exposition only
    };
}
```

An object of type `synchronized_value<T>` wraps an object of type `T`. The wrapped object can be accessed by passing a callable object or function to `apply`. All such accesses are done with a lock held to ensure that only one thread may be accessing the wrapped object for a given `synchronized_value` at a time.

```
template<class ... Args>
synchronized_value(Args&& ... args);
```

Constraints:

`is_constructible_v<T,Args...>` is true

Effects:

Direct-non-list-initializes the contained value with `std::forward<Args>(args)...`.

Throws:

Any exceptions thrown by the selected constructor of `T`.

`std::system_error` if any necessary resources cannot be acquired.

~synchronized_value();

Effects:

Destroys the contained object of type τ and `*this`.

30.x.2 apply function

```
template<class F,class FirstValue,class ... OtherValues>
decltype(auto) apply(
F&& f,synchronized_value<FirstValue>& first_value,synchronized_value<ValueTypes>&... other_values);
```

Effects:

Equivalent to:

```
____ scoped_lock lock(first_value.__mut,other_values.__mut...);
____ return INVOKE(std::forward<F>(f),first_value.__value,other_values.__value...);
```

Returns:

The return value of the invocation of `f`.

Throws:

`std::system_error` if there was an error acquiring any of the locks. Any exceptions thrown by the invocation of `f`.

Synchronization:

Multiple threads may call `apply()` concurrently without external synchronization. If multiple threads call `apply()` concurrently passing the same instance(s) of `synchronized_value` then the behaviour is as-if they each made their call in some unspecified order. The completion of the full expression associated with one invocation of `apply` synchronizes-with a subsequent invocation of `apply` where the same instance of `synchronized_value` is passed to both invocations of `apply`.

Requires:

A single instance of `synchronized_value` shall not be passed more than once to the same invocation of `apply`.

[Example:

```
____ synchronized_value<int> sv;
____ void f(int,int);
____ apply(f,sv,sv); // undefined behaviour, sv passed more than once to same call
```

—End Example]

The invocation of `f` shall not call `apply` directly or indirectly passing any of `first_value`, `other values`....

Changes since P0290R2

Following discussion in LWG in Kona, the following changes have been made:

1. Updated wording to match C++23 draft

Changes since P0290R1

Following discussion in LEWG in Kona, the following changes have been made:

1. Fixed HTML typos;
2. Changed signature of `apply` so it overloads `std::apply` for tuples, and is a better match when a `synchronized_value` is supplied.

Changes since P0290R0

Following discussion in SG1 in Oulu, the following changes have been made:

1. The wording has been changed to allow alternative synchronization mechanisms instead of mutexes;
2. Calling `apply` with the same `synchronized_value` more than once in the same argument list is explicitly disallowed;
3. Recursively calling `apply` with an overlapping set of `synchronized_value` objects is explicitly disallowed; and
4. Constructing a `synchronized_value` may throw `std::system_error`.